

Binary Search

Binary search finds a target in a sorted array by repeatedly cutting the search interval in half.

How It Works (Natural Explanation)

Think of searching in a dictionary: - Open near the middle. - If your word is alphabetically larger, ignore the left half. - If smaller, ignore the right half. - Repeat on the remaining half.

Each step removes about half of the remaining candidates, so the search space shrinks very quickly.

Important precondition: the data must be sorted according to the same comparison rule used in the search.

Formal View

Given a sorted array $A[0..n-1]$ and target x , maintain indices ℓ and r for the active interval.

At each step: - Let $m = \lfloor \frac{\ell+r}{2} \rfloor$. - If $A[m] = x$, return found. - If $A[m] < x$, continue on $[m+1, r]$. - If $A[m] > x$, continue on $[\ell, m-1]$.

Stop when $\ell > r$ (target not present).

Why The Time Complexity Is What It Is

- Best case: $O(1)$
 - The middle element is immediately the target.
- Worst case: $O(\log n)$
 - After each comparison, the remaining interval size is at most half.

If $T(n)$ is worst-case comparisons, then:

$$T(n) = T(n/2) + O(1).$$

After k steps, the remaining size is about $n/2^k$. Stop when this becomes 1:

$$\frac{n}{2^k} \approx 1 \Rightarrow k \approx \log_2 n.$$

So binary search performs logarithmically many comparisons.

When To Use It

- Data is sorted (or can be sorted once and searched many times).
 - Fast lookup is needed.
 - Random access is available (arrays are ideal).
-

Strengths And Limitations

- Strengths:
 - Excellent asymptotic search time on sorted data.
 - Scales much better than linear search for large n .
 - Limitations:
 - Requires sorted order.
 - On linked lists, random middle access is inefficient, so benefits are reduced.
-

Related

- [\[\[Linear Search\]\]](#)
- [\[\[Searching Algorithms\]\]](#)
- [\[\[Lectures/Week 2\]\]](#)